Lab 10

Many intros to tools

Today's plan

00	**Announcements**	
01	**Websites with MyST**	Intro. Credit slides to Fall 2025 STAT 159's staff and Chris Holdgraf
02	**Quarto demo**	
03	**GitHub Actions Workflows**	Intro
04	**Binder**	Intro
05	**Checkpoint**	

Minor update on HW03

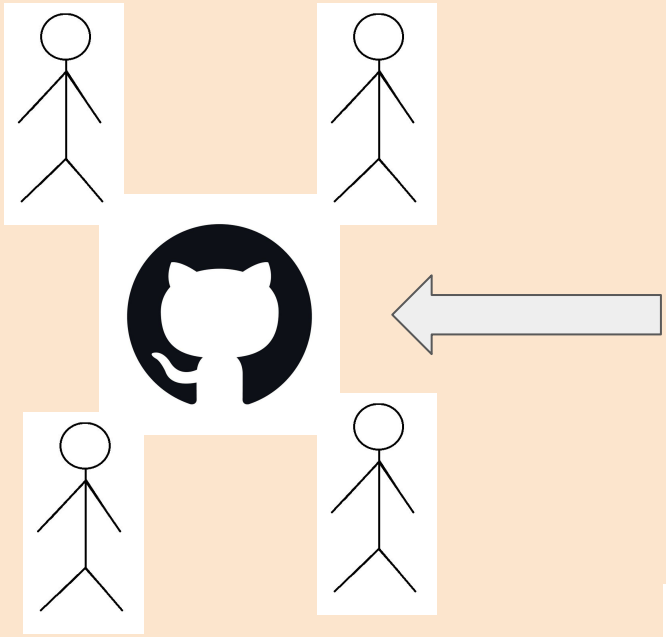
Reflections in `README.md`

Like in Project-2, in your `README.md` file please include a section of personal reflections about the things you learn while working on this assignment.

Intro to MyST

Publishing Computational Projects

Good Practice for Conducting Open Research





notebook1.ipynb



script1.py



script2.py



README.md

- Good documentation (comments, docstrings, readmes)
- Executable notebooks/analyses for reproducibility
- Organized repos (good commit messages etc.)

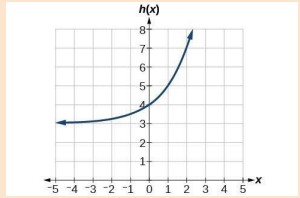
When it comes time to publish..

notebook1.ipynb

script1.py

script2.py

README.md

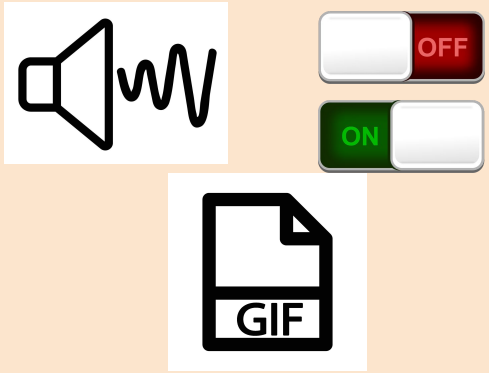


time	n.risk	n.event	survival	std.err	lower 95% CI	upper 95% CI
5	23	2	0.913	0.0588	0.8049	1
8	21	2	0.8261	0.079	0.6848	0.996
9	19	1	0.7826	0.086	0.631	0.971
12	18	1	0.7391	0.0916	0.5798	0.942
13	17	1	0.6957	0.0959	0.5309	0.912
18	14	1	0.646	0.1011	0.4753	0.878
23	13	2	0.5466	0.1073	0.3721	0.803
27	11	1	0.4969	0.1084	0.324	0.762
30	9	1	0.4417	0.1095	0.2717	0.718
31	8	1	0.3865	0.1089	0.2225	0.671
33	7	1	0.3313	0.1064	0.1765	0.622
34	6	1	0.2761	0.102	0.1338	0.569
43	5	1	0.2208	0.0954	0.0947	0.515
45	4	1	0.1656	0.086	0.0598	0.458
48	2	1	0.0828	0.0727	0.0148	0.462

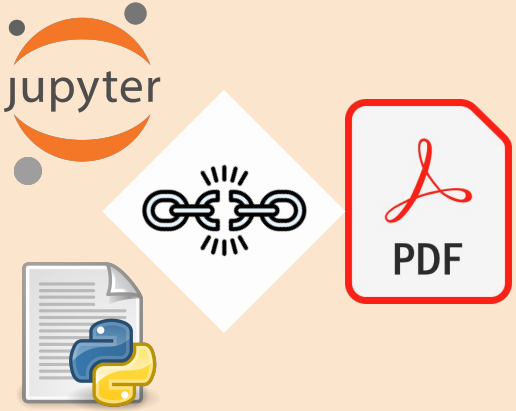
PDF

A shiny publication in your favorite journal!

What are some limitations of this practice?



Limited format to communicate dynamic content *(interactive, audio, animations..)*



Code that generated output untethered from what's in pdf



Not easy to convert to other formats (like interactive webpage)

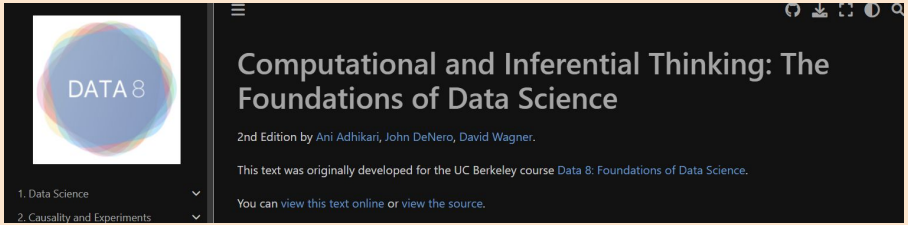
Enter Jupyter Book 2/MyST

First, what is Jupyter Book?

jupyter {book}

“A project that builds tools for authoring, reading, and publishing computational narratives.”

First begun in June 2018. Now used by over 18,000 publicly available books, lectures, and tutorials, including..



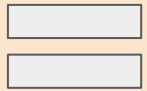
Data 8 Textbook

The Idea...



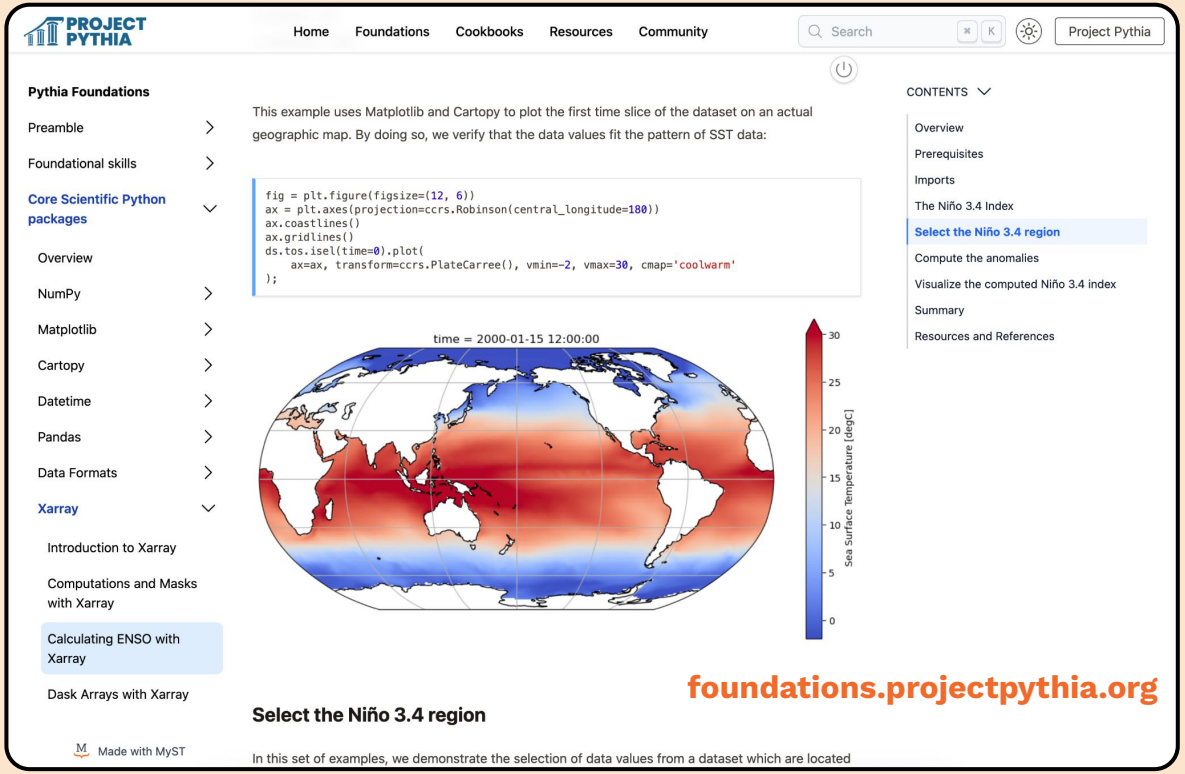


```
> jupyter book build
```



Courtesy: Chris Holdgraf

The Idea...



Courtesy: Chris Holdgraf

Jupyter Book 2 (JB2): Now Built on MyST

MyST (**M**arkedl**y** **S**tructured **T**ext)

- “A collection of open source tools and standards for authoring open computational narratives”
- Produces nice documents and websites *with ease*
- Easily extensible and modular
- Framework very amenable to displaying computational content



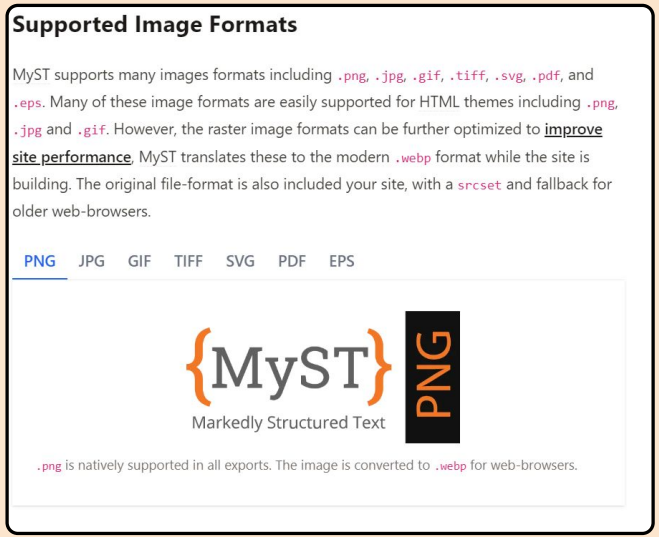
Jupyter Book 2 is a lightweight configuration of MyST for multi-document websites

Courtesy: Chris Holdgraf

MyST Superpower: Content as Structured Data

A MyST document is structured and semantic data.

mystmd.org/guide/figures



Supported Image Formats

MyST supports many images formats including `.png`, `.jpg`, `.gif`, `.tiff`, `.svg`, `.pdf`, and `.eps`. Many of these image formats are easily supported for HTML themes including `.png`, `.jpg` and `.gif`. However, the raster image formats can be further optimized to improve site performance. MyST translates these to the modern `.webp` format while the site is building. The original file-format is also included your site, with a `srcset` and fallback for older web-browsers.

PNG JPG GIF TIFF SVG PDF EPS

{MyST} Markedly Structured Text PNG

`.png` is natively supported in all exports. The image is converted to `.webp` for web-browsers.

mystmd.org/guide/figures.json




```
{
  "kind": "Article",
  "sha256": "eaa6f0ee517c525dfc9db18e6af679fbb43fcd5c6a938ff0703e90c49393b0e2",
  "slug": "figures",
  "location": "/figures.md",
  "dependencies": [],
  "frontmatter": {
    "title": "Images, Figures and Videos",
    "description": "MyST Markdown allows you to create images and figures in yo
    "short_title": "Images, Figures, & Videos",
    "thumbnail": "https://cdn.curvenote.com/019137af-5ae0-7409-9678-66c17851ece
    "license": {
      "content": {
        "id": "CC-BY-4.0",
        "name": "Creative Commons Attribution 4.0 International",
        "free": true,
        "CC": true,
        "url": "https://creativecommons.org/licenses/by/4.0/"
      },
      "code": {
        "id": "MIT",
        "name": "MIT License",
        "osi": true,
        "free": true,
        "url": "https://opensource.org/licenses/MIT"
      }
    },
```


Courtesy: Chris Holdgraf

MyST Superpower: Content as Structured Data

Embed content in many places and hover over references to display context.

methodologies are specifically tailored for efficiency and accessibility for geophysical inverse problems. To create a new `Mesh` instance, a `TensorMesh` class can be selected from the `discretize` module and instantiated with a list of vectors:

```
import discretize  # See Appendix A and Cockett et al. 2016
import numpy as np
import scipy.sparse as sp
hx = np.ones(30)
hy = np.ones(30)
mesh = discretize.TensorMesh([hx, hy])
```

Here, we import the `discretize` library as well as NumPy (`np`) and SciPy's sparse matrix package (`sp`) (Oliphant, 2007; Jones *et al.*, 2001). The vectors `hx` and `hy` describe the cell size in each mesh dimension. The dimension of the mesh is defined by the length of the list, requiring very little change to switch mesh dimensions or type. Once an instance of a mesh is created, access to the properties and methods, shown in Table 1, is possible. Additional methods and visualization routines are also included in the `Mesh` classes. Of note in Table 1 are organizational properties (such as counting and geometric properties), locations of mesh variables as Cartesian grids, differential and averaging operators, and interpolation matrices. We can readily extend the mesh implementation to other types of finite volume meshes (for example, octree (Haber & Heldmann, 2007), logically rectangular non-orthogonal meshes (Hyman *et al.*, 2002), and unstructured meshes (Ollivier-Gooch & Van Altena, 2002)). Additionally, this piece of the framework may be replaced by other methodologies such as finite elements.

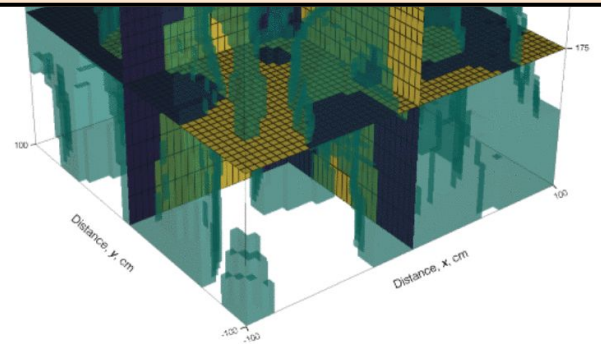


Figure 10: Soil structure in three dimensions showing four sections and the boundary between two soil types of sand (yellow) and loamy sand (purple). The two cross sections and the shallower depth section are shown in subsequent figures.

4.5.1 Results

For calculation of synthetic data, the initial conditions are a dry soil with a homogeneous pressure head ($\psi = -30$cm). The boundary conditions applied simulate an infiltration front applied at the top of the model, $\psi = -10\text{cm} \in \delta\Omega^{\text{top}}$. Neumann (no-flux) boundary conditions are used on the sides of the model domain. Figure 13 shows the pressure head and water content fields from the forward simulation. Figure 11a and 11b show two cross-sections at time 5.2 hours and 10.3 hours of the pressure head field. These figures show true soil type model as an outline, where the inclusions are the less hydraulically-conductive loamy

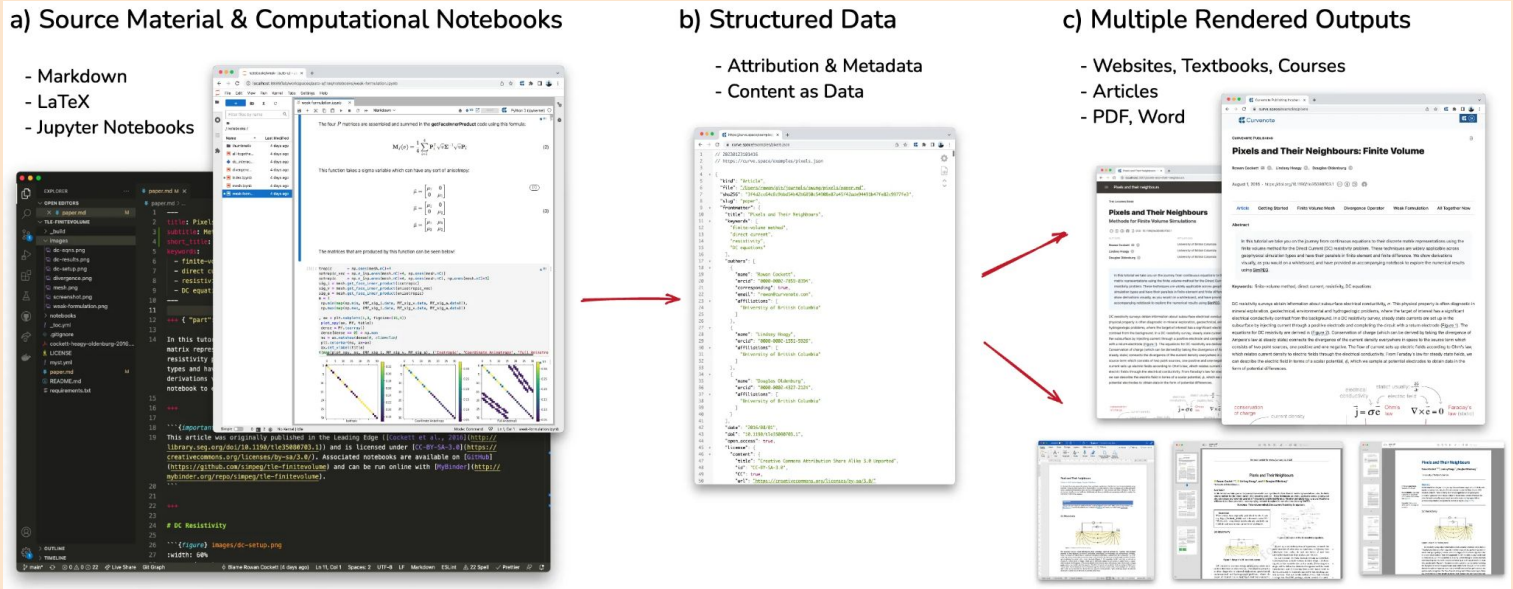
IN THIS ARTICLE

Introduction
Attribution and dissemination
Empirical relationships
Objective functions
Layered soil profile
Unconstrained joint inversion
Results
Discussion
Three dimensional inversion
Results
Scalability of the implicit sensitivity
Discussion
Integrations
Conclusions

Courtesy: Chris Holdgraf

MyST Superpower: Modular and Extensible Design

Structured data means you can publish once and render many times



Courtesy: Chris Holdgraf

MyST Superpower: Modular and Extensible Design

Extend markdown with directives and roles



Courtesy: Chris Holdgraf

MyST Superpower: Modular and Extensible Design

MyST Markdown

````
```{warning} Alert!
Here's a warning!
```
````

````
```{figure} path/to/cool.png
:name: a-label-to-reference
My figure caption!
```
````

```
:::{card} My card title
:header: Here's a header
Check out the [MySTMD
Guide](https://mystmd.org/guide) for
more information!
:::
```

Output

Alert!

Here's a warning!



Fig. 1 My figure caption!

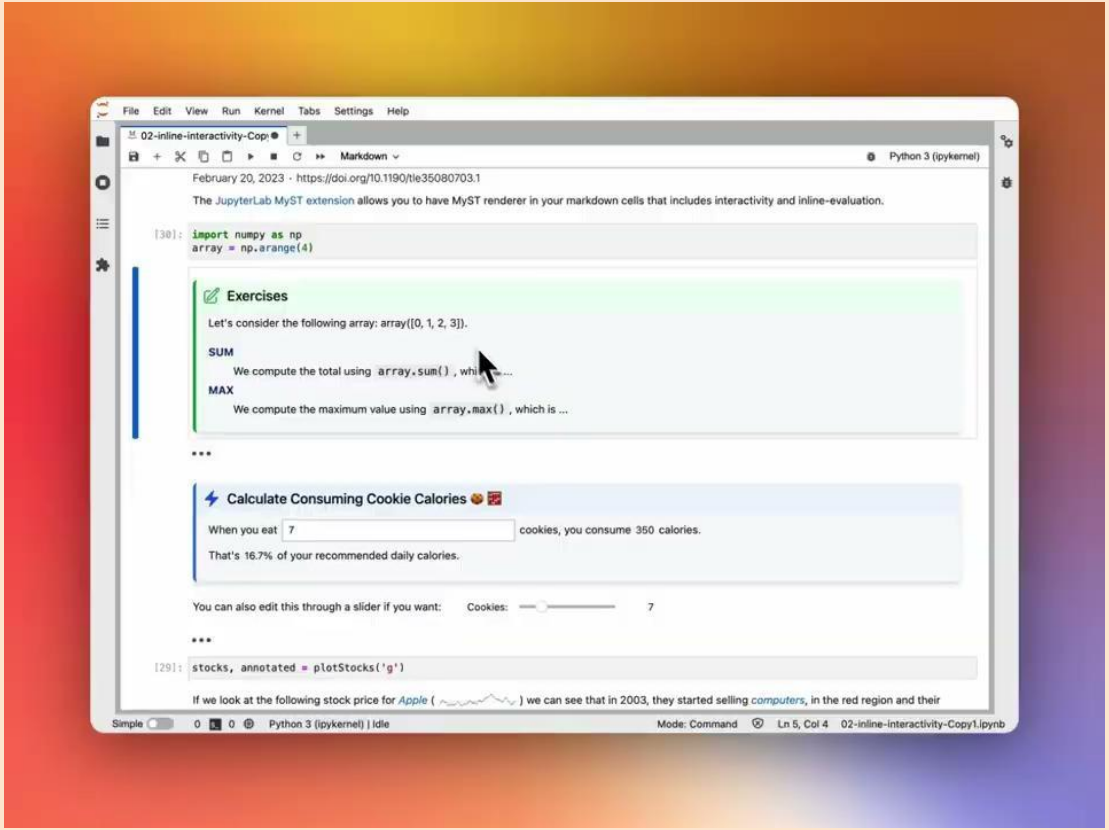
Here's a header

My card title

Check out the MySTMD Guide for more information!

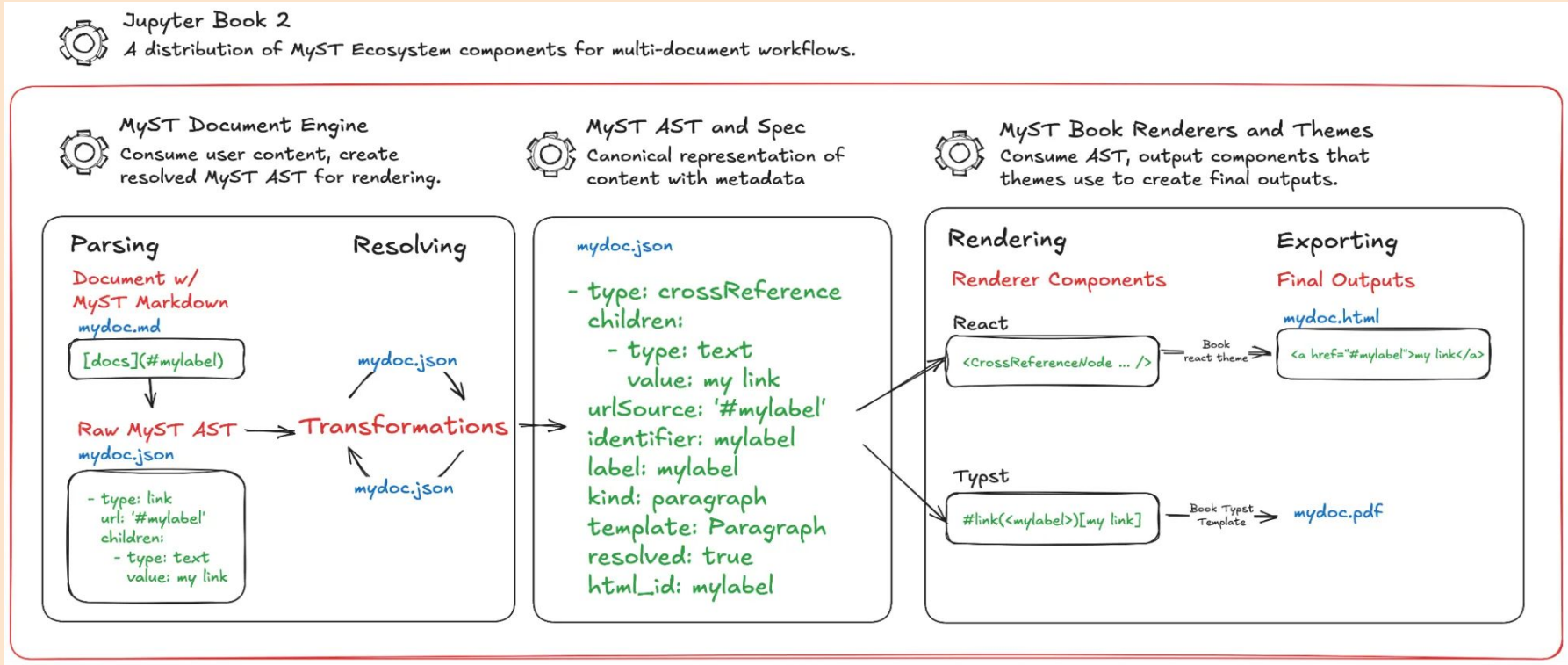
MyST Superpower: Computation is a first class citizen

MyST in JupyterLab: Blending computation and narrative within an interactive environment



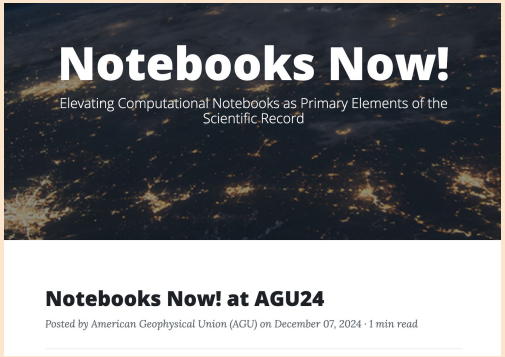
Courtesy: Chris Holdgraf

MyST and JB2: High-Level Components

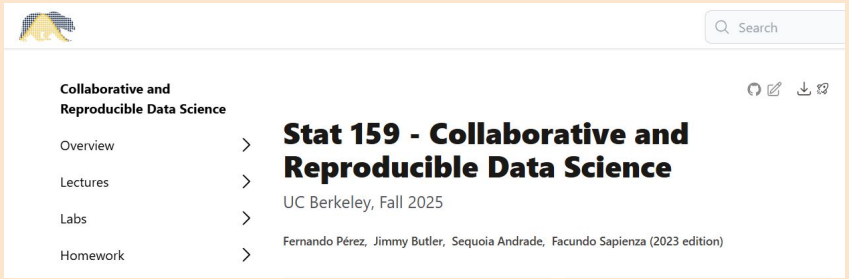


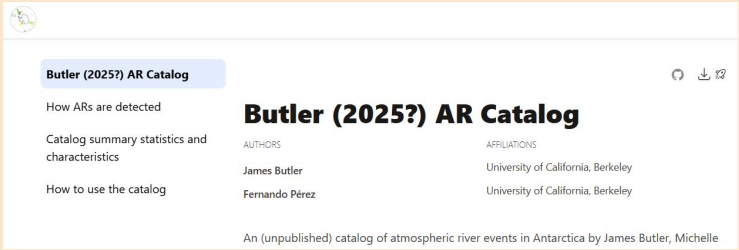
Courtesy: Chris Holdgraf

MyST Use Examples









We will walk through the creation of this website, showing how easy it is!

Demo for MyST

We will be barely scratching the surface of what MyST can do; we will mainly be focused on the mechanics of getting set up, and hopefully you will get your footing by the end of this demo.

- Using the https://github.com/UCB-stat159-f26/lab10 Repo - FORK IT into your account, then clone
- We will be making a MyST site using the lab10 repo. Navigate to it to get started!

Initialize a MyST Project

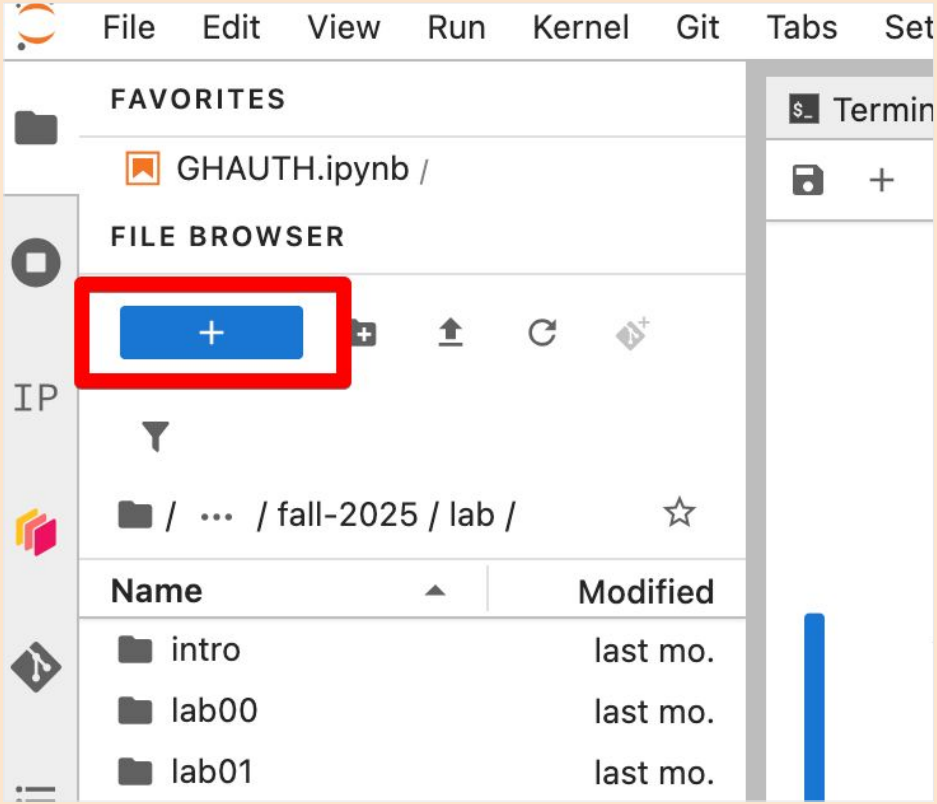
MyST has a simple CLI, that makes it very easy to deploy and build beautiful projects and websites. To start a new MyST project, navigate to the home folder of your repository, and type `myst init` in the command line. You will be prompted if you would like to run myst start. Let's say no for now.

Note

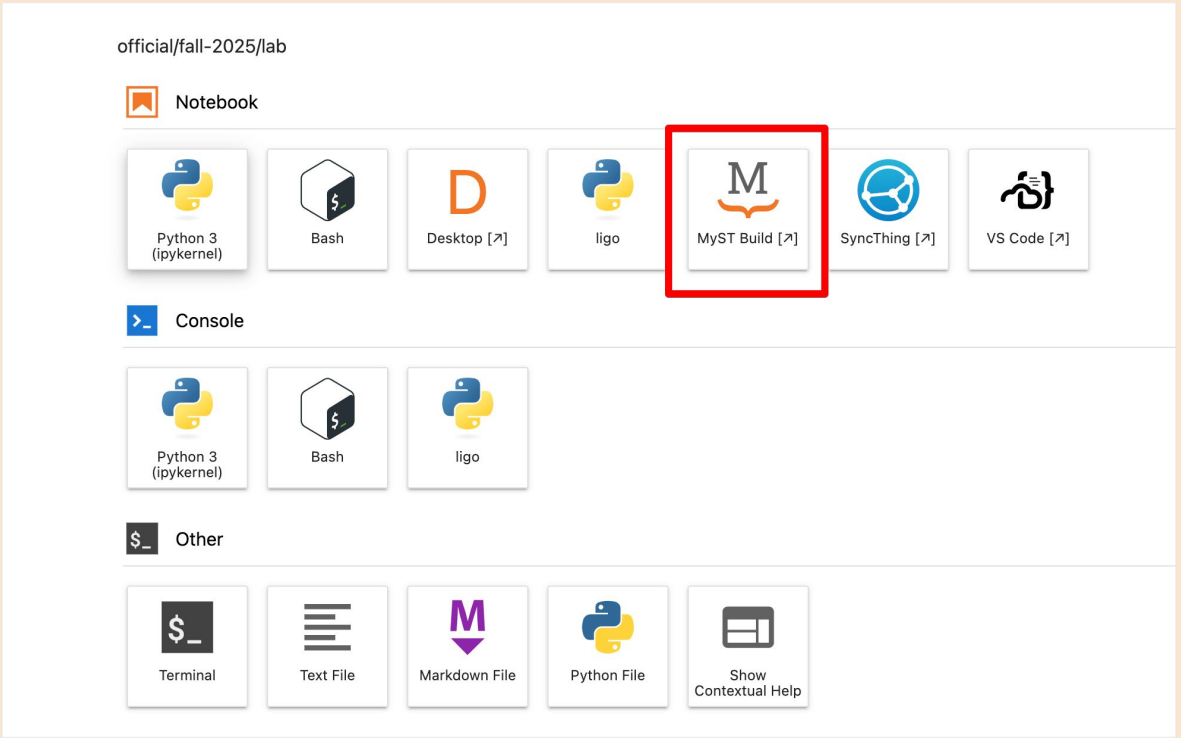
`myst start` is useful to run if you'd like to view a locally hosted version of your site, but it only works if you are using MyST on your local computer. Since we are operating from the Datahub, we'll have to do something else (covered next).

Viewing a local build of a MyST website is very simple!

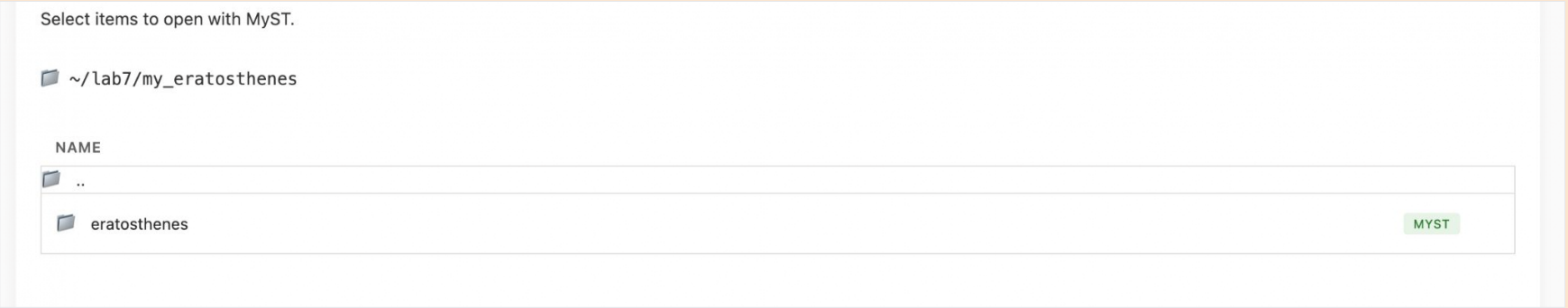
1. Click the blue plus button on the left side of the page to open the launcher.



2. Click MyST Build

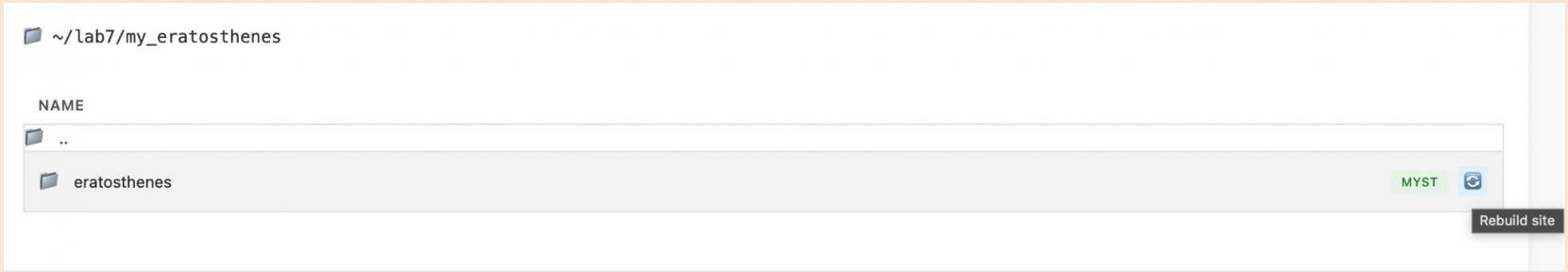


3. A new tab will open. Navigate to the directory that contains your MyST project. *Note: all directories that are MyST projects will be indicated as such*



4. If you've made some changes since your last website build, hover over the directory and click the 'Rebuild Site' icon. If you'd like to look at the version that's already been build, just click the directory

Now, you have a local deployment of your site! Navigate around at your leisure.



MyST Demo (follow along)

MyST demo by GSI is on lab10 repo:
https://github.com/UCB-stat159-f26/lab10/tree/main

Now try both the MyST (notebook 1) & Quarto Demo -> (in your DataHub's _shared folder)

Break

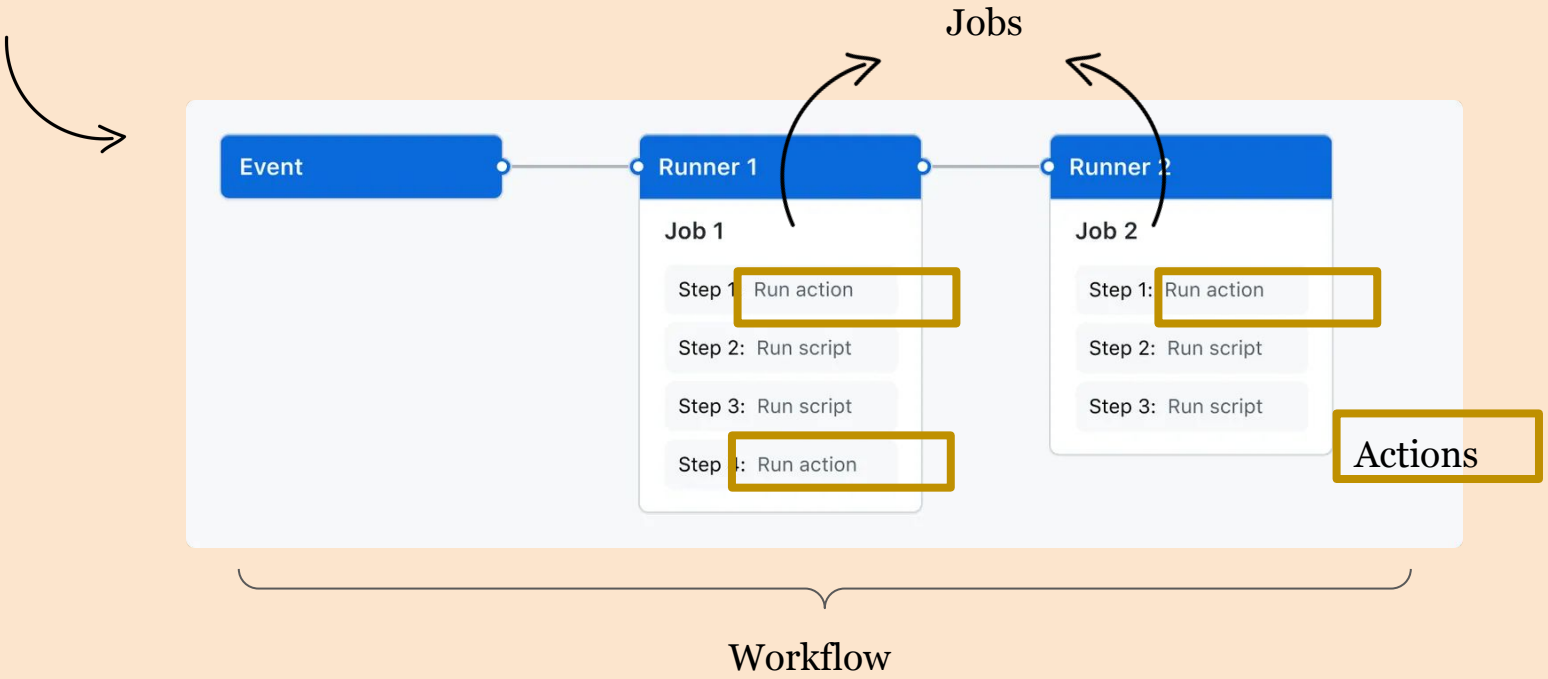
GitHub Actions & Workflows

What's a GitHub Action?

- An Action is a task that runs on GitHub's own servers, automatically, when **something happens** to your repo. They don’t run locally.
- It stemmed from the Continuous Integration/Continuous Delivery (CI/CD) framework, so these actions allow you to automate your build, test, and deployment pipeline.
- **"Something happens"** means there exists a trigger, such as: you push a commit, you open a pull request, or on a schedule (e.g., nightly).

Elements of GitHub Actions

Eg. pull request being opened or an issue being created.



Workflows

A workflow is a configurable automated process that will run one or more jobs.

Workflows are defined by a YAML file that lives in your repo (inside .github/workflows). And will run when triggered by an event in your repository, or they can be triggered manually, or at a defined schedule.

A repository can have multiple workflows, each of which can perform a different set of tasks such as:

- Building and testing pull requests
- Deploying your application every time a release is created
- Adding a label whenever a new issue is opened

Workflows

- Lives at .github/workflows/<name>.yml. Any .yml file in that folder is picked up automatically.
- Four pieces, in order:
  - on: the trigger (push, pull_request, schedule, etc.)
  - jobs: one or more named jobs, each running on a fresh virtual machine (runs-on: ubuntu-latest)
  - steps: an ordered list within a job
  - each step either uses: a pre-built action (someone else's packaged step, e.g. "check out my code") or run: a plain shell command

Here are more Events that trigger workflows

Example of a yaml file

```yaml
name: Run checks
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Say hello
        run: echo "Hello from a GitHub-hosted machine!"
```

Jobs

- A job is a set of steps in a workflow that is executed on the same runner.
- Each step is either a shell script that will be executed, or an action that will be run.
- Steps are executed in order and are dependent on each other.
- Since each step is executed on the same runner, you can share data from one step to another.
    - For example, you can have a step that builds your application followed by a step that tests the application that was built.

Actions

An action is a pre-defined, reusable set of jobs or code that performs specific tasks within a workflow, reducing the amount of repetitive code you write in your workflow files. Actions can perform tasks such as:

- Pulling your Git repository from GitHub
- Setting up the correct toolchain for your build environment

What happens after it runs?

- After a push, go to the repo's **Actions** tab, each run shows every step, with a green check or red X.
- Click into a run to see the actual log output
- A failing Action is useful information: it means something that worked locally didn't work in a clean environment.

Now try notebook 2

Binder

What is Binder?

- It’s a tool that takes a public GitHub repo + an environment file and turns it into a live, interactive computing session anyone can open in a browser without any installs, or accounts required.
- Binder reads the same environment.yml you already know how to write, and builds that exact environment on a temporary cloud machine (a Docker image).

Now, we are going to launch the code into the cloud so everyone can run it!

1. We are going to use **Binder** for this. It's very simple.
2. Go to mybinder.org and enter the information of the repository. Then just enter Launch! In order for this to work, be sure that the environment file needs to be specified under the name environment.yml. Be sure your GitHub repository contains that file.
3. Be sure your repository is public.
4. If everything is in order, this will launch the JupyterLab interface you are already using for this course. Check that the code runs here too and that all the required packages are installed in this new server. Now you can see how Binder helps ensure reproducibility.
5. Binder allows you to create a badge and link that will launch the virtual machine every time you click on it. See how to do this and add this badge to the README.md file.

Try with your lab 10 fork

Reading Checkpoint